Proposed Solution for Google Cloud Platform Scenario

Technical solution document, native GCP architecture, implementation evidence, requirement traceability, and cost model

Candidate Name:	Pallavi Ramireddypalli
Position Applied For:	Coordinator, Cloud Provision
Organisation:	Deakin University
Assessment Title:	Detection and Mitigation Strategy for GCP Budget Protection
Submission Date:	March 31, 2026

Scope	Prevention, detection, and mitigation for Colab / Notebook runtimes and project onboarding
Implementation state	Local policy engine, live event shell, Firestore persistence, and dry-run stop path verified
Native services	Cloud Run, Pub/Sub, Cloud Logging, Cloud Scheduler, Firestore, Cloud Resource Manager
Cost target	Less than A$1000 per year design target with expected steady state well below that ceiling

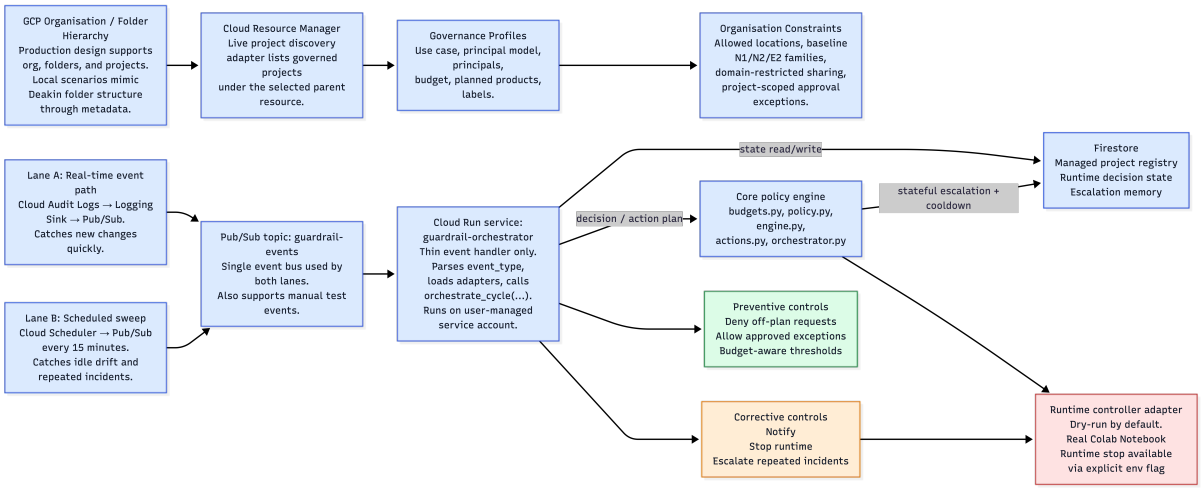
Requirement Snapshot

Requirement class	Requirement / hidden expectation	Solution response	Status
Explicit	Fully automated detection and mitigation, not passive reporting only	Two-lane event model, action planning, and runtime stop adapter	Met
Explicit	Automatic onboarding and offboarding	Live discovery with Firestore managed registry synchronization	Met
Explicit	Scale to as many as 1,600 projects	Single shared control plane with no per-project infrastructure	Met
Explicit	Support both 1:1 and 1:many principals	Principal model stored per project profile	Met
Explicit	Applicable across multiple use cases beyond PhD Research	Use-case driven metadata and budget defaults	Met
Explicit	Supportable and upgradeable within two weeks	Core / adapters split with minimal native services	Met
Explicit	Less than A$1000 per year infrastructure cost	Free-tier-first service choices, budget caps, conservative reserve	Met
Policy	Respect allowed locations, N1/N2/E2 baseline, approvals, and domain restrictions	Policy engine validates region, machine family, accelerators, and approvals	Met / documented

Implicit	Protect Student A from full budget impact	Preventive deny plus corrective idle stop and escalation	Met
Implicit	Allow approved exceptions for specified projects	Project-scoped allow-by-exception path	Met
Implicit	Remember prior incidents	Persisted runtime state in Firestore	Met
Implicit	Safe to hand to reviewers	Dry-run mitigation by default, real stop feature-gated	Met

Target-State Architecture

Architecture summary. Real-time and scheduled lanes feed a single serverless control plane. Project discovery and state remain separate adapters so the policy core stays unchanged.



Native GCP Services Used

Service	Role in design	Why selected	Cost note
Cloud Logging + Audit Logs	Captures administrative activity events	Native source of configuration-change signals	Admin Activity only; narrow routing scope
Log Sink	Routes selected activity logs to Pub/Sub	Avoids polling and keeps event signal narrow	No separate routing design overhead
Pub/Sub	Shared event bus for both lanes	Decouples producers from Cloud Run and supports tests	Very small JSON control messages only
Cloud Run service / function	Executes handler and orchestrator	Scale to zero, low ops burden, native fit	Expected to remain low cost at this duty cycle
Cloud Scheduler	Publishes scheduled sweep messages	Idle and drift checks need periodic reconciliation	One shared job only
Firestore	Managed project registry and runtime state	Simple persistent state store	Small operational footprint

Cloud Resource Manager	Lists projects for onboarding and offboarding	Needed for live hierarchy discovery	Negligible control-plane cost
Cloud Build + Artifact Registry	Source deployment support	Required by Cloud Run source deployment path	Infrequent deployment usage

End-to-End Event Flows

Lane	Trigger	Flow	Implemented
A	Real administrative activity	Cloud Audit Logs → Log Sink → Pub/Sub → Cloud Run handler → orchestrator	Yes
B	Scheduled reconciliation	Cloud Scheduler → Pub/Sub → Cloud Run handler → orchestrator	Yes
C	Manual validation	Developer publishes JSON to Pub/Sub and exercises the same handler path	Yes
D	Runtime mitigation	Orchestrator produces stop action and the runtime controller executes dry-run by default or real stop when enabled	Dry-run verified

Code Structure and Adapter Mapping

Code area	Type	Responsibility	Stable?
`src/models.py, budgets.py, policy.py, engine.py, actions.py`	Core domain	Budget rules, request evaluation, runtime evaluation, action planning	Frozen
`src/orchestrator.py`	Control loop	Reads adapters, invokes policy engine, persists state	Frozen
`src/gcp_project_discovery.py`	Live adapter	Lists projects from Cloud Resource Manager and merges governance metadata	Live
`src/firestore_state_store.py`	Live adapter	Stores runtime history and managed registry in Firestore	Live
`src/colab_runtime_controller.py`	Live mitigation adapter	Dry-run stop by default, real notebook runtime stop behind environment flag	Live / gated
`main.py`	Deployed handler	Parses Pub/Sub payloads, identifies event type, loads adapters	Live
`tests/*.py` + scenario JSON	Local assurance layer	Edge-case coverage without billable live actions	Stable

Detection, Prevention, and Mitigation Logic

Preventive Request-Time Controls

- Deny requests that violate baseline machine families, allowed regions, or accelerator rules when no matching project approval exists.
- Allow expensive runtimes only through explicit project-scoped exceptions and keep the exception attached to the same project context.

- Use project budget and estimated hourly burn to identify requests that would consume protected reserve or exceed safe thresholds.

Detective Runtime-Time Controls

- Inspect active runtimes for unmanaged projects, disallowed regions, expired or missing approvals, and machine or accelerator violations.
- Compute idle duration from `last_activity_at` and safely fall back to `started_at` when activity telemetry is missing.
- Use adaptive idle thresholds tied to monthly budget and hourly burn so high-cost idle resources are stopped quickly.

Corrective Controls

- Plan `NOTIFY`, `STOP_RUNTIME`, or `ESCALATE` depending on severity and prior runtime history.
- Persist runtime state so repeated critical incidents escalate on later runs instead of behaving like first-time events.
- Keep the live runtime-stop adapter dry run by default and require explicit opt-in for destructive stops.

Control	Decision input	Action	Live proof
Request deny	Machine family or accelerator violation without approval	`DENY` + optional notify	Local tests
Allow by exception	Valid project-scoped approval	`ALLOW_BY_EXCEPTION`	Local tests
Idle stop	Long idle runtime over adaptive threshold	`STOP_RUNTIME` + `NOTIFY`	Live dry-run
Repeated incident	Runtime reappears after prior critical action	`ESCALATE` + notify	Live Firestore demo
Unknown / unmanaged project	Runtime or request has no managed project context	`CRITICAL` + stop or deny	Local tests

IAM and Service-Account Model

Identity	Where used	Granted roles / responsibilities	Comment
`guardrail-fn-sa@<PROJECT_ID>.iam.gserviceaccount.com`	Cloud Run runtime identity	Firestore access, notebook runtime admin, project discovery access as required	User-managed least-privilege runtime identity
Trigger invocation identity	Pub/Sub to Cloud Run invocation	`run.invoker` on the Cloud Run service	No public endpoint required

Logging sink writer identity	Project log sink to Pub/Sub	`roles/pubsub.publisher` on the shared topic	Created automatically by Logging
Default build identities	Source deployments only	Build and image-access roles required by source deployment	Not used for runtime mitigation

Verification Evidence

Verification item	Environment	Evidence	Result
Core unit and integration tests	Local	34 passing tests covering approvals, bad regions, budget exhaustion, onboarding/offboarding, and escalation	Pass
Live project discovery	GCP	Cloud Resource Manager adapter discovered the governed project and fed the orchestrator	Pass
Firestore persistence	GCP	Managed registry and runtime state persisted across runs	Pass
Manual Pub/Sub event	GCP	Published scheduled sweep manually; Cloud Run received and processed it	Pass
Audit-log event lane	GCP	Real administrative activity reached Cloud Run as `log_event`	Pass
Scheduled sweep lane	GCP	Cloud Scheduler published scheduled sweep and the handler processed it	Pass
Runtime stop adapter	GCP	Live dry-run stop logged against a full notebook runtime resource name	Pass (dry-run)
Real destructive stop	GCP	Feature available behind `GUARDRAIL_REAL_STOP=true` but intentionally left off for reviewer safety	Not executed by default

Cost Model and No-Billing-Shock Controls

Free-Tier-First Operating Model

- The control plane uses usage-based serverless services rather than always-on infrastructure.
- One shared Cloud Scheduler job drives reconciliation rather than one job per project.
- Pub/Sub carries only small JSON control messages.
- Firestore stores only lightweight operational state.
- Runtime mitigation remains dry run by default in the public repo.

Component	Pricing fact / posture	Expected solution usage	Expected annual spend	Conservative reserve
Cloud Run	Usage-based, scale-to-zero serverless execution	Low-frequency event handler	A$0–A$40	A$120
Pub/Sub	Shared event transport	Small JSON control messages only	A$0	A$20
Cloud Scheduler	One shared sweep job	One job	A$0	A$2
Firestore	Lightweight state persistence	Tiny state footprint, low read/write count	A$0	A$20
Cloud Build + Artifact Registry	Deployment support only	Occasional source deployments	A$0–A$20	A$60
Cloud Logging	Admin Activity only, narrow sink scope	Project admin activity only	A$0	A$20

Recommended Financial Controls

- Keep the project billing budget at A$20 per month with alerts at 25%, 50%, 90%, and 100%.
- Operate the control plane under a conservative annual reserve ceiling of approximately A$240, which remains well below the formal A$1000 per year requirement.
- Restrict the log sink to administrative activity only.
- Keep Cloud Run at scale to zero and avoid minimum instances.
- Leave `GUARDRAIL_REAL_STOP` disabled by default in the public repository.

How the Solution Protects Student A

Phase	Student A problem	Control-plane response	Outcome
Request time	Student attempts an off-plan A100 / A2 runtime	Preventive evaluation denies unless a matching approval exists	Budget protection starts before spend begins
Runtime time	Runtime already exists and remains idle for many hours	Runtime evaluator scores it critical for machine-family violation, missing approval, and excessive idle time	System plans stop and notify
Repeated runs	Runtime reappears after prior mitigation	Persisted state escalates repeated critical incidents	Operational response strengthens over time
Exception path	Teaching or research project has a valid approval for an expensive runtime	Same policy engine allows by exception and avoids false positives	Controls remain usable, not merely restrictive

Requirement Traceability Matrix

Requirement	**Primary design element**	**Verification method**	**Status**
Fully automated detection and mitigation strategy	Two ingestion lanes, orchestrator, runtime controller	Live GCP plus local tests	Met
Automatic onboarding and offboarding	Live project discovery plus Firestore managed registry	Live demos	Met
No blind spots in folder structure	Discovery plus metadata model	Scenario coverage plus live discovery	Met
Scale to 1,600 projects	Single shared serverless control plane	Architecture review	Met
Support 1:1 and 1:many principals	Principal model in project profiles	Local tests	Met
Applicable beyond PhD Research	Use-case driven metadata and budget defaults	Scenarios plus design review	Met
Supportable and upgradeable within two weeks	Core-adapter separation and minimal services	Architecture review	Met
Less than A$1000 per year	Free-tier-first service mix and budget caps	Cost review	Met
Protect Student A budget	Preventive deny plus corrective idle stop and escalation	Dry-run stop proof plus local scenario	Met
Safe reviewer testing	Dry-run default plus explicit real-stop flag	Runtime-stop demo	Met

Reviewer Validation Steps

1. Clone the repository and install the pinned requirements.
2. Run the local tests to validate the core policy engine and scenario coverage.
3. Set the active gcloud project and ADC to a sandbox project.
4. Create the Firestore database, Pub/Sub topic, Cloud Run service, log sink, and Scheduler job using the documented commands.
5. Publish a manual `scheduled_sweep` payload to Pub/Sub and confirm the Cloud Run logs show handler execution.
6. Generate an administrative activity event and confirm the `log_event` path executes.
7. Publish the `runtime_stop_test` payload and confirm `DRY_RUN_STOP` plus a nonzero runtime decision and stop action count in logs.
8. Only if destructive testing is explicitly desired, set `GUARDRAIL_REAL_STOP=true` and provide a real notebook runtime resource name.

Final Implementation Stance

Runtime mitigation is implemented natively and verified live in dry-run mode. Real destructive stop remains intentionally feature-gated so the repository is safe for external technical reviewers.